🔷 1. What is Optimization in ML?

📌 Definition

Optimization is the process of finding parameters that minimize (or maximize) a function.

In ML:

$$\theta^* = \arg\min_{\theta} L(\theta)$$

Where:

- $L(\theta)$ = loss function
- θ = model parameters

🎯 What It Does

Optimization:

- Minimizes loss
- Updates weights
- Trains neural networks
- Improves predictions

🤖 Why It Matters

Without optimization:

- Model never learns
- Weights never update
- Loss never decreases

🔷 2. Gradient Descent (Foundation)

🔹 2.1 Idea

Move in direction of steepest descent.

Update rule:

$$\theta = \theta - \alpha \nabla L(\theta)$$

Where:

- α = learning rate
- $\nabla L(\theta)$ = gradient

🔹 2.2 Intuition

Imagine standing on a hill:

- Gradient → slope direction
- Move opposite → go downhill

🔹 2.3 Types of Gradient Descent

Type	Data Used
Batch GD	Entire dataset
SGD	One sample
Mini-batch GD	Small batch

🔷 3. Stochastic Gradient Descent (SGD)

📌 Definition

Updates parameters using one data point at a time.

$$\theta = \theta - \alpha \nabla L_i(\theta)$$

🎯 What It Does

- Faster updates
- Noisy but efficient
- Escapes local minima

🤖 Why Used in Deep Learning?

- Large datasets
- Memory efficient
- Faster convergence

🔹 Code Example

🔹 Problem with SGD

- Oscillates
- Noisy updates
- Sensitive to learning rate

Solution → Momentum methods.

🔷 4. Momentum

📌 Idea

Add memory of previous gradients.

Velocity update:

$$v_t = \beta v_{t-1} + \nabla L(\theta)$$

Parameter update:

$$\theta = \theta - \alpha v_t$$

Where:

- $\beta \approx 0.9$

🎯 Why Important?

- Smooth updates
- Faster convergence

- Reduces oscillation

🧠 Intuition

Like rolling a ball:

- Gains speed downhill
- Smooth motion

🔷 5. RMSprop

📌 Problem It Solves

SGD struggles when gradients vary across dimensions.

📌 Idea

Scale learning rate using squared gradients.

$$s_t = \beta s_{t-1} + (1-\beta) g_t^2$$

Update:

$$\theta = \theta - \frac{\alpha}{\sqrt{s_t} + \epsilon} g_t$$

🎯 What It Does

- Adaptive learning rate
- Larger steps in flat regions
- Smaller steps in steep regions

🧠 Why Important?

Great for:

- Recurrent Neural Networks
- Non-stationary problems

🔷 6. Adam (Adaptive Moment Estimation)

🔥 Most popular optimizer.

Combines:

- Momentum
- RMSprop

📌 Step 1: Compute Gradient

$$g_t = \nabla L(\theta_t)$$

📌 Step 2: First Moment (Momentum)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

📌 Step 3: Second Moment (RMSprop)

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

📌 Step 4: Bias Correction

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

📌 Step 5: Update Rule

$$\theta = \theta - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

🎯 Why Adam Works So Well

- Adaptive learning rates

- Smooth updates
- Works well out-of-the-box
- Good for sparse gradients

🔹 Code Implementation (From Scratch)

Minimizing:

$$L(w) = w^2$$

🔷 7. Comparison Table

Optimizer	Speed	Stability	Adaptive LR	Best For
SGD	Slow	Low	❌	Simple models
Momentum	Faster	Medium	❌	Deep networks
RMSprop	Fast	High	✅	RNNs
Adam	Very Fast	Very High	✅	Most DL tasks

🔷 8. When NOT to Use Adam?

Recent research shows:

- Sometimes SGD generalizes better
- Adam may overfit

For vision models:

- SGD + Momentum often best

🔷 9. Learning Rate Scheduling

Common techniques:

- Step decay
- Exponential decay
- Cosine annealing

- Warm restarts

Very important for convergence.

🔷 10. Convergence & Saddle Points

Neural networks:

- Non-convex
- Many saddle points

Momentum & Adam help escape.

🧠 PRACTICE PROBLEMS

🟢 Beginner

1. Write SGD update for:
   $L(w) = w^2$
2. Why does large learning rate cause divergence?

🟡 Intermediate

3. Derive momentum update rule.
4. Explain bias correction in Adam.

🔴 Advanced

5. Prove Adam combines RMSprop + Momentum mathematically.
6. Compare convergence behavior of SGD vs Adam.
7. Show why adaptive methods may hurt generalization.

🎯 INTERVIEW QUESTIONS

---

1. Why is Adam popular?

Because it adapts learning rate per parameter.

---

2. What does beta1 control?

Momentum (gradient smoothing).

---

3. What does beta2 control?

Variance scaling.

---

4. Why add epsilon?

Avoid division by zero.

---

5. Why can SGD generalize better?

Because noise in updates helps escape sharp minima.

🚀 FINAL BIG PICTURE

Training Neural Network =

Linear Algebra → Compute forward pass
Calculus → Compute gradients
Optimization → Update parameters
Probability → Model uncertainty

Optimization algorithms are the engine of learning.

Next deep options:

🔥 Backpropagation Full Derivation
🔥 Neural Network Mathematics Deep Dive
🔥 Convex Optimization Theory

- 🔥 Regularization (L1, L2, Dropout math)
- 🔥 Transformer Optimization Dynamics

Choose your next level 🚀